

شرح احترافي ومفصل لكود Audio Emotion Extraction

تحليل بنية ال Notebook وشرح وظيفة كل جزء داخل خط سير العمل

هذا المستند يشرح النوتبوك بالكامل: من تحميل بيانات CREMA-D وتجهيزها، إلى معالجة ملفات الصوت، واستخراج الخصائص، واختيار ال features، ثم تدريب نموذج SVM ومقارنة النتائج النهائية. كل بلوك كود هنا موضوع داخل مستطيل واضح كما طلبت.

الفكرة العامة للمشروع

الهدف هو بناء نظام Speech Emotion Recognition يستطيع تصنيف المشاعر من ملف صوتي. الفئات التي يتعامل معها المشروع هي: Angry, Disgust, Fear, Happy, Neutral, Sad. ويقوم النوتبوك بتنفيذ pipeline كاملة تشمل تحميل البيانات، بناء ال metadata، التحقق من سلامة الملفات، المعالجة المسبقة، استخراج الخصائص، إزالة القيم الشاذة، معالجة القيم الناقصة، التحجيم، تقسيم البيانات، اختيار الخصائص، ثم التدريب والتقييم.

إعداد البيئة وتحميل البيانات

في البداية يتم تجهيز بيئة Colab وتهيئة الوصول إلى Kaggle. ثم يتحقق الكود من وجود ملف kaggle.json، ويحفظه في المسار القياسي حتى تتمكن أوامر Kaggle API من العمل. بعد ذلك يثبت مكتبات مثل librosa وdeap و tqdm. الفكرة الأساسية هنا هي جعل النوتبوك قادراً على تنزيل بيانات CREMA-D وتشغيل جميع مراحل التحليل بدون اعتماد يدوي كبير.

إذا كان مجلد AudioWAV موجوداً بالفعل، يتخطى الكود مرحلة التنزيل، وإلا يبدأ التنزيل من Kaggle أو يقبل رفع الملف المضغوط يدوياً. هذه مرونة جيدة لأنها تجعل النوتبوك صالحاً لأكثر من بيئة تشغيل.

بناء ال Metadata من أسماء الملفات

بعد استيراد المكتبات الأساسية، يتم تعريف خريطة تربط كود المشاعر بالاسم الكامل لكل شعور. هذا الجزء محوري لأنه يحول الابل النصي إلى تمثيل عددي يمكن للنماذج التعامل معه.

```

{ = EMOTION_CODE_MAP
  , 'Angry' : 'ANG'
  , 'Disgust' : 'DIS'
  , 'Fear' : 'FEA'
  , 'Happy' : 'HAP'
  , 'Neutral' : 'NEU'
  , 'Sad' : 'SAD'
}
(values.EMOTION_CODE_MAP(enumerate in v ,i for i :v{ = LABEL_TO_INT
})
)items.LABEL_TO_INT in v ,k for k :v{ = INT_TO_LABEL

```

بعدها يقرأ الكود اسم كل ملف ليستخرج معلومات مثل actor_id و emotion_code و intensity و filepath و يجمعها في جدول meta_df. هذا الجدول هو المرجع الأساسي لباقي المراحل.

التحقق من سلامة ملفات الصوت

قبل بدء التحليل الحقيقي، يختبر الكود إمكانية فتح الملفات الصوتية. إذا كان الملف مفقوداً أو تالفاً أو فارغاً، يتم استبعاده من البيانات. هذه الخطوة تمنع فشل العمليات اللاحقة وتحسن جودة البيانات الداخلة إلى مرحلة استخراج الخصائص.

المعالجة المسبقة للصوت

يحدد الكود قيماً أساسية مثل SAMPLE_RATE = 16000 و MIN_DURATION = 0.5 و TRIM_DB = 20. ثم يعرف دالة مسؤولة عن تجهيز كل ملف صوتي قبل التحليل.

```

:tuple >- )str :filepath(preprocess_audio def
    """
    .file audio single a preprocess and Load

    :steps Preprocessing
    )kHz (16 rate sample fixed at Load 1.
    mono to Convert 2.
    silence trailing/leading Trim 3.
    duration minimum Check 4.
    """

```

وظيفة هذه الدالة أنها توحد صيغة الإشارة الصوتية، وتزيل الصمت من البداية والنهاية، وتستبعد المقاطع القصيرة جداً. هذا

مهم لأن النموذج لا يستفيد من اختلافات تقنية بين الملفات بقدر ما يستفيد من المحتوى العاطفي نفسه.

استخراج الخصائص الصوتية

قلب المشروع موجود في دالة استخراج الخصائص. بدلاً من تمرير الموجة الصوتية الخام إلى النموذج، يحولها الكود إلى متجه رقمي غني بالمعلومات الطيفية والزمنية.

```
:ndarray.np >- )str :filepath(extract_features def
    """
wav. a from vector feature acoustic dimensional-162 a Extract
    .file

    :breakdown vector Feature
80 = (40)          mfcc_std + (40)          mfcc_mean
40 = (20)  mfcc_delta_std + (20)  mfcc_delta_mean
24 = (12)          chroma_std + (12)          chroma_mean
    ...
    """
```

هذه الخصائص تشمل MFCC و MFCC Delta و Chroma وخصائص طيفية مثل spectral centroid و bandwidth و rolloff، بالإضافة إلى zero crossing rate و RMS و pitch وملخصات إحصائية من mel spectrogram.

ملاحظة مهمة: التعليق داخل الدالة يقول إن عدد الخصائص 162، لكن التنفيذ الفعلي في النوتبوك يصل إلى 200 خاصية. هذا يتأكد من الملخص النهائي الذي يعرض Feature dim = 200. لذلك يوجد تعارض بين ال docstring والتنفيذ الحقيقي.

تنظيف مصفوفة الخصائص

بعد استخراج المتجهات العددية من جميع الملفات، يزيل الكود العينات غير الطبيعية. من أمثلة ذلك وجود NaN أو Inf أو أن يكون المتجه كله أصفاراً أو ذا norm مرتفع جداً بشكل شاذ. بعد ذلك يعالج القيم الناقصة. إذا كان الصف يحتوي على نسبة كبيرة جداً من القيم المفقودة فإنه يُحذف، وإلا يتم تعويض القيم المفقودة باستخدام median لكل عمود. اختيار median منطقي هنا لأنه أكثر ثباتاً من mean في الخصائص التي قد تكون منحرفة التوزيع مثل بعض تقديرات pitch.

التحجيم وتقسيم البيانات

يبني الكود pipeline للتحجيم تتكون من StandardScaler ثم Normalizer(norm='l2'). الأول يجعل الخصائص على مقياس موحد، والثاني يطبع كل عينة. بعد ذلك تُقسم البيانات إلى تدريب وتحقق واختبار بنسب تقريبية 70/10/20 مع استخدام stratify للحفاظ على توزيع الفئات.

النقطة الجيدة جداً هنا أن أداة التحجيم يتم عمل fit لها على بيانات التدريب فقط، ثم تطبيقها على التحقق والاختبار. بذلك يتجنب الكود مشكلة data leakage.

تقييم أي مجموعة خصائص

قبل اختيار الخصائص، يعرف النوتبوك دالة تقيس جودة أي مجموعة مختارة باستخدام نموذج KNN على مجموعة التحقق.

```
(evaluate_feature_subset def
,ndarray np :X_train
,ndarray np :y_train
,ndarray np :X_val
,ndarray np :y_val
,ndarray np :feature_mask
5 = int :k
:float >- )
".accuracy classification KNN using subset feature a Evaluate""
"
```

هذه الدالة هي fitness function التي تعتمد عليها كل من الطريقة الجشعة والخوارزمية الجينية. اختيار KNN هنا سببه أنه سريع وبسيط، وبالتالي مناسب للتكرار مرات كثيرة أثناء البحث.

الاختيار الجشع للخصائص

الطريقة الجشعة Greedy Forward Selection تبدأ من مجموعة فارغة، ثم في كل خطوة تضيف أفضل خاصية تحقق تحسناً على validation accuracy. هذا الأسلوب سهل التفسير، لكنه قد يقع في local optimum لأنه يتخذ قراراً محلياً في كل مرة.

```
(greedy_forward_selection def
    ,ndarray.np :X_train
    ,ndarray.np :y_train
    ,ndarray.np :X_val
    ,ndarray.np :y_val
    ,30 = int :max_features
    ,5 = int :k_nn
    True = bool :verbose
    :tuple >- )
as accuracy KNN using Selection Feature Forward Greedy"""
""".fitness
```

في التجربة المعروضة، هذه الطريقة اختارت حوالي ٢٥ خاصية.

الخوارزمية الجينية لاختيار الخصائص

بعد ذلك ينتقل النوتبوك إلى خوارزمية جينية Genetic Algorithm. هنا تمثل كل مجموعة خصائص بـ كروموسوم ثنائي: 1 يعني أن الخاصية مستخدمة و 0 يعني أنها غير مستخدمة. ثم تُبنى أجيال متتالية من الحلول باستخدام selection, elitism, mutation, crossover.

```
(genetic_algorithm_feature_selector def
    ,ndarray.np :X_train
    ,ndarray.np :y_train
    ,ndarray.np :X_val
    ,ndarray.np :y_val
    ,20 = int :n_population
    ,10 = int :n_generations
    ,7.0 = float :cx_prob
    ,2.0 = float :mut_prob
    ,0.2 = float :gene_flip_prob
    ,2 = int :elite_size
    ,5 = int :k_nn
    :)
...

```

هذه الطريقة أوسع في البحث من الطريقة الجشعة، ولذلك غالباً تكون أقوى، لكنها تحتاج وقتاً أطول. في هذا النوتبوك انتهت إلى اختيار نحو ٨٥ خاصية.

تدريب نموذج SVM والتقييم النهائي

بعد تجهيز ثلاث نسخ من البيانات - جميع الخصائص، والخصائص المختارة بالطريقة الجشعة، والخصائص المختارة بالخوارزمية الجينية - يدرب الكود نموذج SVM واحداً بنفس الإعدادات على كل نسخة.

```
(train_and_evaluate_svm def
,ndarray.np :y_tr ,ndarray.np :X_tr
,ndarray.np :y_te ,ndarray.np :X_te
str :name
:tuple >- )
,'scale'=gamma ,0.10=C , 'rbf'=kernel(SVC = clf
=42)random_state , 'ovr'=decision_function_shape
```

يحسب بعدها Accuracy, Precision, Recall, و F1 Macro. والأخير مهم لأنه يعطي كل فئة وزناً متساوياً في التقييم.

النتائج

الجدول التالي يلخص المقارنة النهائية بين الإعدادات الثلاثة:

الإعداد	عدد الخصائص	Accuracy	Macro F1
Baseline	٢٠٠	٥٥١٨.٠٠	٥٤٧٢.٠٠
(Greedy) Heuristic	٢٥	٤٨١٢.٠٠	٤٧٤٣.٠٠
Algorithm Genetic	٨٥	٤٨٩٩.٠٠	٤٨٥٥.٠٠

النتيجة الواضحة هي أن استخدام جميع الخصائص كان الأفضل من حيث الأداء النهائي على SVM. وهذا يعني أن تقليل عدد الخصائص هنا لم يحسن الدقة، رغم أنه قلل الأبعاد.

أهم ملاحظة منهجية

اختيار الخصائص في النوتوك تم باستخدام KNN كدالة fitness، بينما التقييم النهائي تم باستخدام SVM. هذه نقطة منهجية مهمة جداً، لأن الخوارزمية كانت تبحث عن خصائص مناسبة لـ KNN وليس بالضرورة لـ SVM. ولهذا السبب أضاف النوتوك لاحقاً تقييماً مصححاً باستخدام SVM cross-validation، وكانت النتيجة ما تزال في صالح Baseline.

الاستنتاج العلمي الأهم: إذا كان النموذج النهائي هو SVM، فمن الأفضل أن تكون عملية اختيار الخصائص مبنية على معيار تقييم قريب من SVM أيضاً، وإلا قد يحدث عدم توافق بين مرحلة الاختيار ومرحلة التقييم النهائي.

نقاط القوة والتحسينات

من نقاط القوة في الكود: التنظيم الجيد، ومنع data leakage، واستخدام cache لتقليل الزمن، وإضافة تقييم تصحيحي لاحق. أما أهم التحسينات المقترحة فهي تحديث وصف عدد الخصائص، وإزالة الاستيرادات غير المستخدمة، وتجربة آلية اختيار خصائص متوافقة مع SVM.

الخلاصة النهائية

هذا النوتبوك قوي جداً كتمارين تطبيقي وتعليمي في Speech Emotion Recognition. فهو لا يقدم مجرد نموذج، بل يوضح دورة عمل كاملة من البيانات الخام حتى المقارنة النهائية. وأهم نتيجة فيه أن أفضل أداء تحقق عند استخدام جميع الخصائص (200)، بينما لم تعطِ استراتيجيات اختيار الخصائص أفضلية على SVM داخل هذا الإعداد.

الخلاصة التنفيذية: المشروع ممتاز من حيث البنية والتنظيم واستخراج الخصائص، لكن مرحلة اختيار الخصائص تحتاج مواءمة أفضل مع النموذج النهائي حتى تعطي فائدة حقيقية في الأداء.